

CM3065 Intelligent Signal Processing Coursework – Midterms 2024

Student Name: Devon Maya Xavier

UOL Student ID: 200618744

SIM Student ID: 10204322

Email: dmxavior001@mymail.sim.edu.sg

Exercise 1: Web based Audio Application

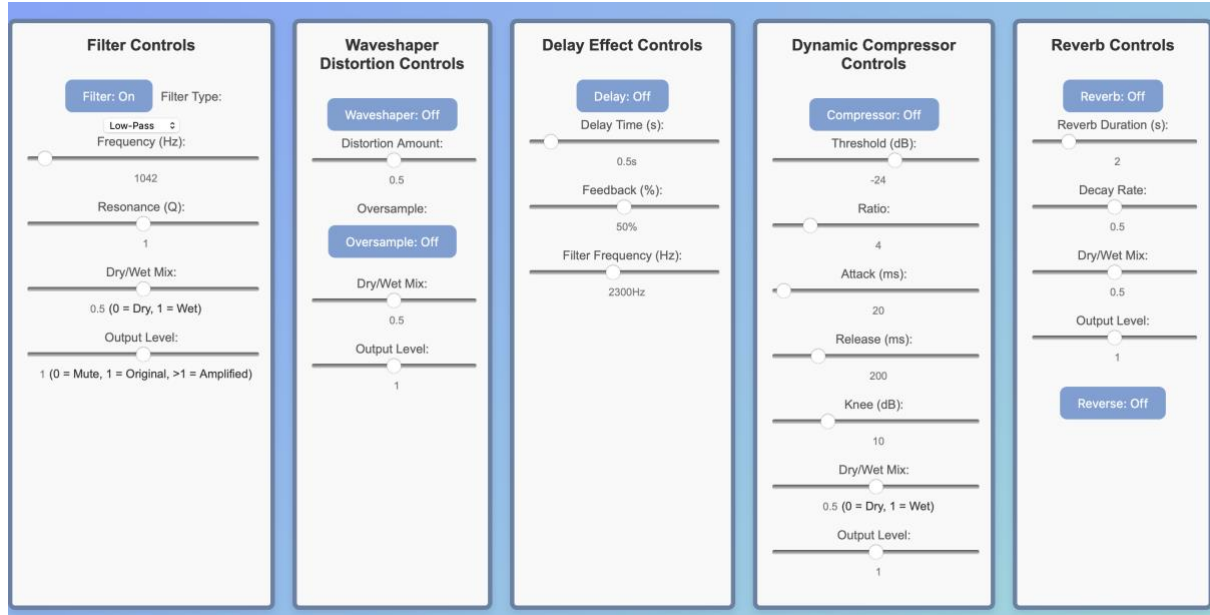
Task 1.1: Web Application with p5.js

<https://hub.labs.coursera.org/connect/sharednetotwda?>

A. Main Characteristics and Programming



Playback controls with Record and Master Volume.



Low-Pass Filter:

This filter allows low frequencies to pass while attenuating higher ones. Users can dynamically adjust the type, cutoff frequency, resonance (Q-factor), and dry/wet mix via HTML controls linked to the p5.Filter object. It can be toggled on or off, ensuring seamless integration into the audio chain.

Waveshaper:

The waveshaper introduces harmonic distortion by applying a non-linear function to the signal. Parameters such as distortion amount, oversampling mode, dry/wet mix, and output level are adjustable through the p5.Distortion object, offering flexibility for sound design.

Delay:

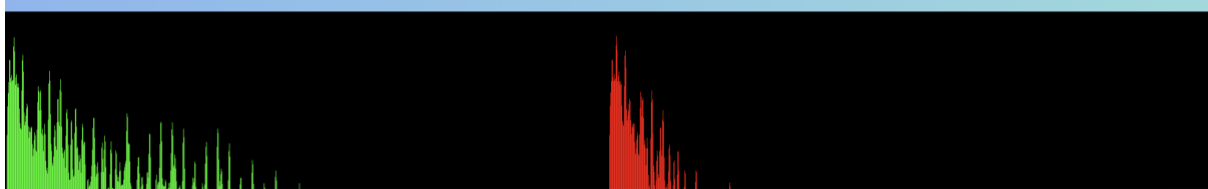
This effect adds echoes by repeating the signal after a specified delay time. Users can fine-tune delay time, feedback, and low-pass filter frequency for the echoes using the p5.Delay object, enabling precise control over the effect.

Dynamic Compressor:

The compressor reduces the dynamic range of the audio signal, ensuring consistent volume by attenuating loud parts and amplifying soft ones. Adjustable parameters include threshold, ratio, attack, release, knee, dry/wet mix, and output level, implemented through the p5.Compressor object.

Reverb:

Reverb adds depth and ambiance by simulating natural sound reverberation in physical spaces. Users can adjust parameters such as reverb duration, decay rate, dry/wet mix, and reverse reverb toggling using the p5.Reverb object.



Spectrum In (Green) and Spectrum Out (Red).

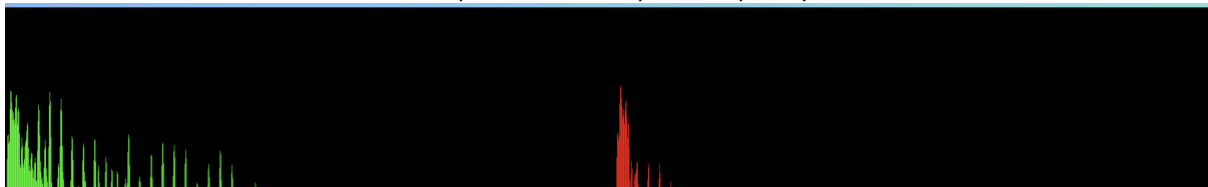
B. Analysis of Low-Pass Filter and Master Volume Effects**Low-Pass Filter:**

The low-pass filter affects the sound spectrum by attenuating high frequencies based on the cutoff frequency and resonance. Lower cutoff frequencies mute higher harmonics, creating a muffled sound, while increased resonance amplifies frequencies near the cutoff, adding a peak to the spectrum. This effect is dynamically visualized using FFT displays for “Spectrum In” and “Spectrum Out.”

Master Volume:

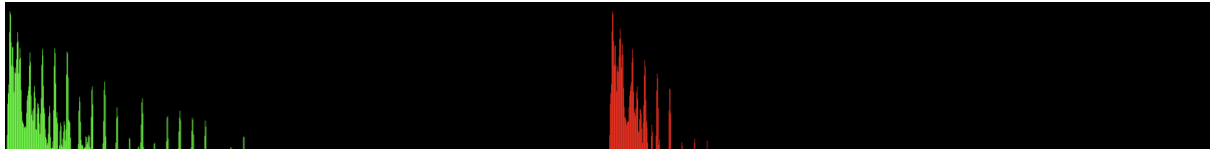
The master volume adjusts the output amplitude with perceptually balanced quadratic smoothing, avoiding abrupt loudness changes. It uniformly reduces or amplifies all frequencies, preserving the spectrum’s balance while managing overall loudness.

Illustration: Low-Pass filter’s effect on Spectrum – Low-pass Frequency at 1000Hz



Without Filter: Full frequency spectrum visible. **With Low-Pass Filter:** High frequencies are attenuated.

Illustration: Master Volume’s effect on Spectrum - With Low-pass Frequency at 1000Hz



Without Filter: Spectrum looks the same as before. **With Master Volume turned up:** Spectrum effects are bigger.

Task 1.2: Enhanced filter effects

C. Analysis of Enhanced Filters

Filter Type Selector:

Originally implemented with the `p5.LowPass` object, the filter was upgraded to the `p5.Filter` object to support low-pass, high-pass, and band-pass filtering. This change allows users to dynamically select the filter type during playback and adjust parameters such as cutoff frequency, resonance, and dry/wet mix in real-time. The enhancement broadens the filter's use, enabling diverse applications such as noise reduction, frequency isolation, and creative effects.

Audio Source Selector:

The option to toggle between live microphone input and pre-recorded audio significantly enhances versatility. To ensure a smoother experience, I set the default microphone amplification level to 0.1. Without this adjustment, the microphone feedback would have been excessively sharp and loud.

Delay Effect Integration:

The delay effect enriches the audio by adding echoes and rhythmic patterns. Adjustable parameters include delay time, feedback, and a low-pass filter to shape the tone of echoes. Positioned before the dynamic compressor, the delay effect ensures dynamic preventing of volume spikes. This feature enhances the application's capability to create immersive soundscapes, from subtle ambiance to dramatic rhythmic effects.

Exercise 2: Audio Captcha and Visualisation

Task 2.1: Audio Captcha

<https://hub.labs.coursera.org/connect/sharedjbadkjq?>

In order to create an effective audio CAPTCHA system that challenges automated systems while remaining intelligible to humans, I've decided on a method that utilised the `p5.speech` library to combine pre-recorded audio files with synthesized scrambled text.

The `scrambledText` function introduces complexity by adding spaces between characters in the CAPTCHA text, disrupting word flow. This scrambled text is then vocalized using the `speech.speak` method, with additional effects like `setPitch` and `setRate` applied to randomize pitch and playback speed. These effects create a distorted and unpredictable output, making it harder for automated recognition systems to interpret.

```

// Apply scrambling effects to the audio
function applyAudioScramble() {
  if (!captchaAudio || !captchaAudio.isLoaded()) {
    console.error("Captcha audio is not loaded or undefined.");
    return;
  }

  // Initialize speech synthesis
  let speech = new p5.Speech(); // p5.Speech instance
  speech.setLang('en-US'); // Set language
  speech.setVolume(1); // Set volume to max

  // Apply random voice from available voices
  const voices = speech.listVoices();
  if (voices && voices.length > 0) {
    const randomVoice = voices[Math.floor(Math.random() * voices.length)];
    speech.setVoice(randomVoice.name);
  }

  // Apply random pitch
  const randomPitch = random(0.5, 1.5); // Adjust pitch range as needed
  speech.setPitch(randomPitch);

  // Apply random rate
  const randomRate = random(0.8, 1.2); // Adjust rate range as needed
  speech.setRate(randomRate);

  // Generate text for speech synthesis
  const scrambledText = captchaText.split('').join(' '); // Introduce spacing between letters

  // Set a softer volume for speech synthesis
  speech.setVolume(0.2); // Set volume to 20% for softer output

  // Speak the scrambled text with effects
  speech.speak(scrambledText);
}

```

The original pre-recorded audio, containing the CAPTCHA sentence, is preserved and played alongside the scrambled speech synthesis. This dual-layer approach ensures humans can distinguish the correct words in the original audio while automated systems struggle with the overlaid distortions. The original audio is minimally modified, maintaining clarity for human listeners while ensuring that the scrambled text remains an intentional distraction. This implementation was tested with speech recognition tools, confirming its effectiveness.

Type in the sentence you hear.


 Refresh Captcha

where are the restaurants and shops

Submit

 **Success! Captcha matched.**

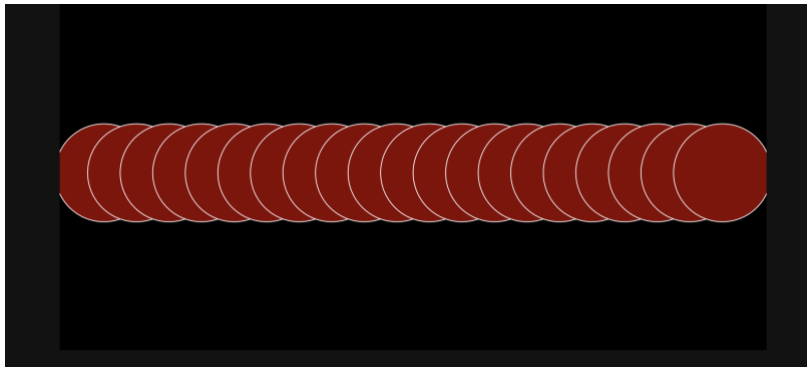
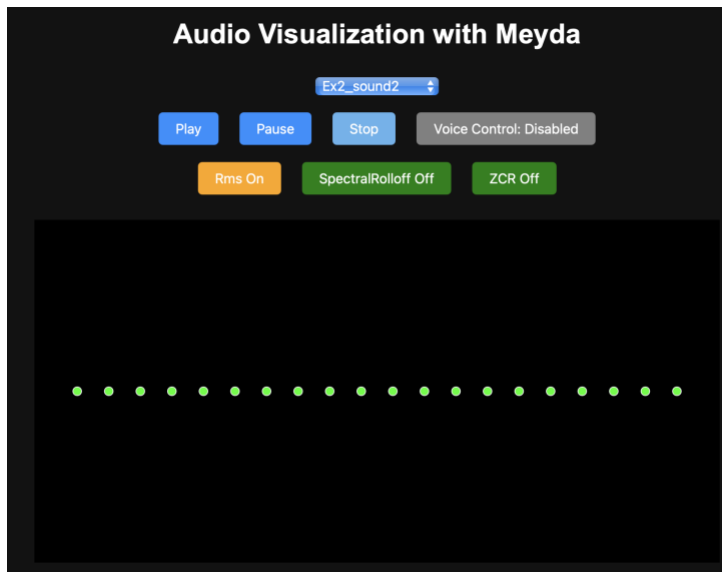
But, to strengthen the ability of the audio captcha, adversarial machine techniques can be added to identify and prevent automated bots from trying to solve the captcha. Also, instead of one full audio file, snippets of files can be played sequentially to ask user for combined phrase or identification of correct sequence of text heard in the sounds. Lastly, visual captcha can be added on top of the audio captcha to enhance robustness of the captcha (i.e., distorted text image that matches parts of the audio etc)

Task 2.2: Audio Visualisation with Meyda

Please refer to files in submitted zip, there was only one Link generator for Ex 2 but I have two web apps

The three Meyda audio features selected for each sound and its justifications:

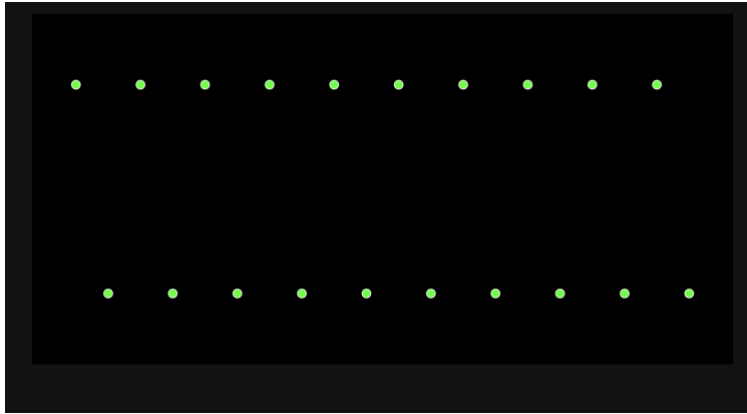
	Meyda Audio Features	Justification
Sound 1	Spectral Rolloff	Spectral Rolloff was good for this sound because it was able to capture the high and low frequencies changes and reflect those changes effectively through the thickness of the circle border and the ripple effect, the higher the frequency, the thicker the border and strength of ripple.
Sound 2	RMS (Root Mean Square)	RMS (Root Mean Square) is ideal for Sound 2's gradual volume build-up, as it effectively captures the audio's energy and dynamic range. It provides smooth, continuous amplitude measurements, making it perfect for visualizing gradual volume changes. RMS also allows intuitive mapping to visual elements like height and brightness, creating a clear and compelling representation of the sound's characteristics.
Sound 3	ZCR (Zero Crossing Rate)	Sound 3 has sharp, rapid transitions in frequency and amplitude, highly dynamic sounds. ZCR is sensitive to such transitions, making it an ideal metric to quantify and visualize these changes. Although ZCR is usually used for detecting speech frame, I found it worked quite well for sound 3.



For RMS, I've chosen to represent its effect through the size and colour of the circles. As the RMS values increase, the circles grow larger and transition in colour from green to red, visually indicating higher intensity.



Spectral Rolloff influences the border thickness of the circles, with thicker borders corresponding to higher values.

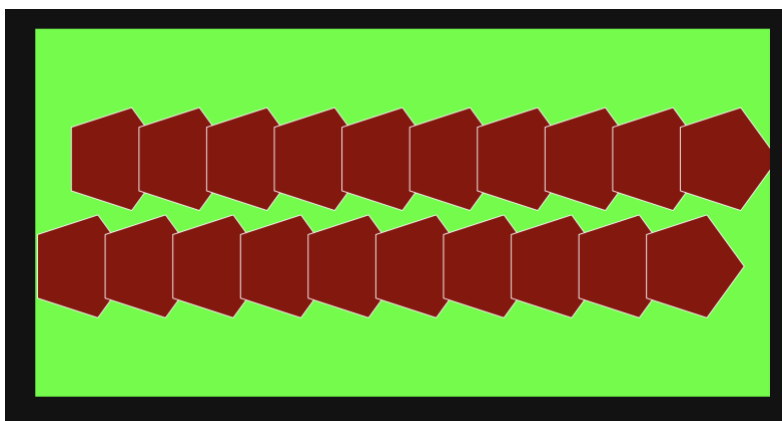


Meanwhile, ZCR impacts the speed and velocity of the bouncing circles.

The reason above designs were chosen is because I believe they are the most intuitively friendly visuals that could translate the dynamic changes in audio signals to the viewers. The **size and colour of the circles** tied to RMS (Root Mean Square) values deliver an immediate visual cue about the intensity of the audio. Larger, redder circles symbolize higher RMS values, aligning with the natural association of size and red hues with strength and intensity. The **border thickness** linked to Spectral Rolloff emphasizes the presence of higher frequencies in the audio. Thicker borders visually stand out against the background, signifying audio segments with a more dominant high-frequency component. This feature ensures that even subtle spectral changes in the audio are made visually prominent. The **speed and velocity** variations influenced by ZCR (Zero-Crossing Rate) create a dynamic representation of the audio's activity. Faster-moving circles with higher ZCR values reflect a more frequent change in the audio signal's polarity, resonating with the idea of a high-energy or chaotic audio environment.

The **black background** was chosen to maximize contrast and visual clarity. It ensures that all elements—circle sizes, colours, and movement—are vividly noticeable, making the visual representation easy to interpret. The **default green colour** for the circles was selected for its association with balance and neutrality, providing a baseline that contrasts well with red for higher RMS values, thereby enhancing the visual narrative. Green also contrasts sharply against the black background, ensuring the visuals remain engaging and accessible to the viewer.

Task 2.3: Voice Controller



To implement the voice control feature, the **p5.SpeechRec** function was utilized to recognize and process basic spoken commands for changing the background colour and the shapes of objects dynamically. Initially, the system encountered a slight delay in executing these changes, often resulting in multiple commands being misinterpreted or overlapping. This was resolved by enabling the **voiceRec.interimResults** property,

allowing the system to provide real-time updates as the user spoke, rather than waiting for the user to pause or stop speaking. This enhancement significantly improved responsiveness and ensured accurate recognition of fast-paced commands.

The integration of **interim results** proved especially beneficial when paired with the audio input from “Kalte_Ohren,” a fast-tempo track. The system’s ability to swiftly respond to voice commands amplified the interactive aspect of the performance, making it visually engaging and responsive to rapid user input. Quick transitions in background colour and shape added an element of immediacy, creating an immersive and stimulating visual display.

The final implementation delivered a visually captivating and interactive performance, combining real-time voice control with audio-responsive animations, showcasing the synergy between speech recognition, audio analysis, and creative visualization.

Exercise 3: Audio Analysis and Steganography

<https://hub.labs.coursera.org/connect/sharedmacszacr?>

Task 3.1: Audio File Analysis

The first task involved analysing four audio files to identify which contained a secret code embedded using amplitude modulation in the ultrasonic frequency range. In order to filter out which sound file contained the secret code, Fourier Transform was utilised to isolate frequencies beyond the human audible range (>20 kHz). The algorithm successfully identified **Ex3_sound1.wav** as the suspicious file containing significant ultrasonic content. By demodulating the ultrasonic frequencies to the audible range, the secret four-number code **1127** was deciphered.

Results:

The secret code was accurately extracted by mapping the detected frequencies to their corresponding numerical values, leveraging a predefined frequency-to-number mapping.

```
# Define a mapping of frequencies to numbers
frequency_to_number = {
    261.63: 0, # C
    293.66: 1, # D
    329.63: 2, # E
    349.23: 3, # F
    392.00: 4, # G
    440.00: 5, # A
    493.88: 6, # B
    523.25: 7, # C
    587.33: 8, # D
    659.25: 9 # E
}

# Decipher the secret code from detected frequencies
secret_code = []
for freq in detected_frequencies:
    # Find the closest frequency in the mapping
    closest_freq = min(frequency_to_number.keys(), key=lambda x: abs(x - freq))
    secret_code.append(frequency_to_number[closest_freq])

# Print the deciphered secret code
print("Deciphered Secret Code:", secret_code)
```

Deciphered Secret Code: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 7]

In order to ensure the rest of the sound files do not contain any embedded code, I tested them as well and the results confirmed sound1 was the only suspicious containing a code.

[illegible]

Task 3.2: Embedding Hidden Messages

Description:

In this task, the Least Significant Bit (LSB) method was implemented for audio steganography to embed and extract secret messages within audio files. LSB steganography modifies the least significant bit of each audio sample to encode the binary representation of a message. This approach ensures minimal perceptual distortion while embedding the message.

Embedding Algorithm:

Ex3_sound5.wav was read and its sample frames were converted into a writable array. The text message to be converted was then converted to a binary sequence and checked for length to see if it could be embedded in the audio data. Once it is confirmed to be of right length, it was embedded into the least significant bits of the audio, and then written out as a new audio file.

```
def embed_message(audio_file, message, output_file):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        params = wav.getparams()
        frames = wav.readframes(params.nframes)

    # Convert frames to numpy array and create a writable copy
    audio_data = np.frombuffer(frames, dtype=np.int16).copy() # Use .copy() to make it writable

    # Convert message to binary
    message_bin = ''.join(format(ord(i), '08b') for i in message)
    message_length = len(message_bin)

    # Check if the message can fit in the audio data
    if message_length > len(audio_data):
        raise ValueError("Message is too long to fit in the audio file.")

    # Embed the message into the least significant bits
    for i in range(message_length):
        audio_data[i] = (audio_data[i] & ~1) | int(message_bin[i])

    # Write the modified audio data to a new file
    with wave.open(output_file, 'wb') as wav_out:
        wav_out.setparams(params)
        wav_out.writeframes(audio_data.tobytes())

    # Input secret message and generate embedded file
    secret_message = "An eye for an eye makes the whole world blind"
    embed_message('Ex3_sound5.wav', secret_message, 'Ex3_sound5_embedded.wav')
```

Extraction Algorithm:

The embedded audio file had its frames converted to numpy array, then its least significant bits were extracted. The binary sequence was then converted back to its alphanumeric representation, resulting in the embedded secret message.

```

def extract_message(audio_file, message_length):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        frames = wav.readframes(wav.getnframes())

    # Convert frames to numpy array
    audio_data = np.frombuffer(frames, dtype=np.int16)

    # Extract the message from the least significant bits
    message_bin = ''
    for i in range(message_length):
        message_bin += str(audio_data[i] & 1)

    # Convert binary to string
    message = ''.join(chr(int(message_bin[i:i+8], 2)) for i in range(0, len(message_bin), 8))
    return message

# Extraction of secret code
extracted_message = extract_message('Ex3_sound5_embedded.wav', len(secret_message) * 8)
print("Extracted Message:", extracted_message)

Extracted Message: An eye for an eye makes the whole world blind

```

Results:

The embedding and extraction algorithms were tested with another audio file and message, the result was successful. Utilising the LSB method, the embedded message is not perceptible to the human ear, as confirmed by the playback testing. Message extracted matched the original as well.

```

# Input secret message and generate embedded file
secret_message = "Someday, everything will make perfect sense. So for now, laugh at the confusion, smile through the tears, and keep reminding
embed_message('Ex3_sound4.wav', secret_message, 'Ex3_sound4_embedded.wav')

def extract_message(audio_file, message_length):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        frames = wav.readframes(wav.getnframes())

    # Convert frames to numpy array
    audio_data = np.frombuffer(frames, dtype=np.int16)

    # Extract the message from the least significant bits
    message_bin = ''
    for i in range(message_length):
        message_bin += str(audio_data[i] & 1)

    # Convert binary to string
    message = ''.join(chr(int(message_bin[i:i+8], 2)) for i in range(0, len(message_bin), 8))
    return message

# Extraction of secret code
extracted_message = extract_message('Ex3_sound4_embedded.wav', len(secret_message) * 8)
print("Extracted Message:", extracted_message)

Extracted Message: Someday, everything will make perfect sense. So for now, laugh at the confusion, smile through the tears, and keep remindi
ng yourself that everything happens for a reason

```

Task 3.3: Further investigation of state-of-the-art audio steganographic approaches

Phase Coding

Phase coding is an audio steganography technique that conceals secret information within an audio file by subtly altering the phase of small segments of the sound wave, which dictates how the wave aligns and progresses over time. Note, only the phase of the first segment is adjusted, while the relative differences between the remaining segments are maintained. This ensures that the changes are so subtle that it is virtually impossible for the human ear to distinguish.

To safeguard the hidden information, a stream cipher layer is added. This combines the secret data with a sequence of random numbers (or a “keystream”) using simple XOR (exclusive OR) operations. This scrambles the data so that it is not readable without the correct key. This also ensures that if someone tries to extract the hidden data, they will not understand anything unless they have the encryption key.

Hence, phase coding has high imperceptibility as the modifications to the audio are minimal. Sound quality remains almost the same as original to human ears. With the dual-layered security system, it gives high

level of security as attackers still need the encryption key to understand the code. It is extremely beneficial for secure communications, digital watermarking, covert data storage etc.

However, there are several limitations as well. The amount of data that can be hidden is relatively small compared to other steganography methods as phase coding prioritises imperceptibility over data volume. Also, the addition of encryption and phase integrity requires high computational overhead, making real-time applications very challenging. Furthermore, it is vulnerable to signal processing, if the audio goes with significant modifications, such as compression or filtering, the embedded data might be corrupted. Lastly, specialised tools like advanced forensic analysis using phase-sensitive tools may still be able to identify anomalies in the audio file.

Comparison with Least Significant Bit (LSB)

Feature	Phase coding	LSB
Imperceptibility	High, as phase changes are undetectable	Medium, detectable with noise analysis
Robustness	Strong against attacks due to encryption	Weak, vulnerable to extraction attacks
Embedding Capacity	Limited by phase constraints	High, depending on audio size
Computational Cost	High, due to FFT/IFFT and encryption	Low, simple bit replacement
Use Cases	To secure communication	For low-security needs

In conclusion, phase coding offers superior imperceptibility and robustness, making it ideal for high-security steganographic applications. While computationally demanding, it surpasses the LSB method in protecting hidden data from detection and extraction, providing a state-of-the-art solution for secure audio steganography.

Exercise 4

<https://hub.labs.coursera.org/connect/sharedmacszacr?>

For the airport virtual assistant, Mozilla's DeepSpeech was chosen as the core offline ASR system. DeepSpeech's ability to operate without internet dependency aligns with the client's requirement to ensure uninterrupted operation in an airport environment. Pre-trained scorer files were utilized for English, Spanish, and Italian, enabling language-specific recognition.

The airport's noisy environment posed a significant challenge to achieving high accuracy in speech recognition. To address this, the following enhancements were implemented:

Zero-Crossing Rate (ZCR) Filtering: ZCR filtering was applied to distinguish between noise and speech. Low ZCR signals were amplified to boost weak audio signals, while high ZCR signals, indicative of noise, were attenuated. This improved audio clarity and recognition accuracy.

Text Normalization: Special characters, accents, and punctuation, particularly prevalent in Spanish and Italian datasets, initially led to high Word Error Rates (WER). Text normalization techniques, such as Unicode normalization, tokenization, and case standardization, were introduced. These changes drastically improved recognition accuracy by ensuring uniformity in transcriptions.

Attempts to add a **band-pass filter** for isolating human voice frequencies (300–3400 Hz) inadvertently worsened the WER, possibly due to the filter altering essential speech characteristics. As a result, the filter was omitted in favour of ZCR filtering and text preprocessing.

The system was tested on client-provided audio files and two additional recordings (your_sentence1.wav and your_sentence2.wav), with the latter two passing at 0% WER.

Recognition Results:

Language	File lab	WER (%)
English	Audio/EN/checkin.wav	0.00
English	Audio/EN/parents.wav	20.00
English	Audio/EN/suitcase.wav	0.00
English	Audio/EN/what_time.wav	20.00
English	Audio/EN/where.wav	0.00
English	Audio/EN/your_sentence1.wav	0.00
English	Audio/EN/your_sentence2.wav	0.00
Italian	Audio/IT/checkin_it.wav	25.00
Italian	Audio/IT/parents_it.wav	20.00
Italian	Audio/IT/suitcase_it.wav	0.00
Italian	Audio/IT/what_time_it.wav	100.00
Italian	Audio/IT/where_it.wav	42.86
Spanish	Audio/ES/checkin_es.wav	25.00
Spanish	Audio/ES/parents_es.wav	0.00
Spanish	Audio/ES/suitcase_es.wav	0.00
Spanish	Audio/ES/what_time_es.wav	83.33
Spanish	Audio/ES/where_es.wav	42.86

Accuracy: English performed better overall, likely due to the extensive dataset available for training the English model. Italian and Spanish recognition showed slightly higher WER, especially with regional accents and noisy recordings.

Phonetic Confusion: Pronunciation similarities, such as “have” vs. “had” or “pancone” vs. “bancone,” occasionally caused errors. These issues underscore the need for fine-tuning or training the model on domain-specific datasets, such as airport-specific terminology and accents.

To explore alternatives, two state-of-the-art ASR systems—OpenAI’s **Whisper** and Meta AI’s **Wav2Vec2**—were evaluated.

OpenAI Whisper:

Strengths: Whisper excels in transcription accuracy and is robust against accents and background noise, thanks to its training on 680,000 hours of diverse data. It demonstrates exceptional performance in handling multilingual tasks and complex environments, such as airports.

Limitations: Whisper primarily functions online, requiring substantial computational resources for real-time offline transcription. This reliance on high-performance hardware may pose a challenge for the airport assistant’s scalability.

Meta AI Wav2Vec2:

Strengths: Wav2Vec2 leverages self-supervised learning to train on unlabelled audio data, making it highly adaptable for noise-rich environments like airports. Its fine-tuning capabilities allow for better handling of accents and domain-specific language. Moreover, Wav2Vec2 can be deployed offline with proper configuration.

Limitations: Wav2Vec2 requires significant computational resources during the training phase, which may impact the time and cost of implementation.

While DeepSpeech provides a functional offline solution, its limitations in handling noise and complex accents suggest the need for improvement. For the airport virtual assistant’s future implementation:

Even though **Whisper** offers remarkable accuracy and noise handling, it may require substantial hardware upgrades to meet offline requirements. Therefore, **Wav2Vec2** is more recommended for its superior transcription accuracy, adaptability to noise, and offline capability. Its fine-tuning potential ensures it can be tailored to the airport's diverse linguistic demands.

References:

Yadnya, M. S., Kanata, B., & Anwar, M. K. (2022). *Using phase coding method for audio steganography with the stream cipher encrypt technique*. Electrical Engineering Department, University of Mataram, Indonesia.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Aspell, A., Mishkin, P., Clark, J., Krueger, G., & Amodei, D. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. OpenAI.

Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). *wav2vec 2.0: A framework for self-supervised learning of speech representations*. Meta AI Research. Retrieved from <https://ai.meta.com/research/publications/wav2vec-2.0-a-framework-for-self-supervised-learning-of-speech-representations/>